# TD machine 1

## Résoudre des PL et PLNE avec le solveur gurobi

Le but de ce TD est d'utiliser le solveur gurobi (à partir du langage python) pour résoudre des problèmes de programmation linéaire. Gurobi est un produit commercial et serait le meilleur solveur pour ce type de problèmes que ce soit avec des variables continues, entières ou binaires ou mixtes [1] mais aussi pour la programmation quadratique (fonction objectif quadratique et contraintes linéaires et/ou quadratiques) et récemment il semble que l'on puisse résoudre des problèmes plus généraux encore. Il est disponible gratuitement pour l'éducation et la recherche (il faut de demander une licence sur le site).

En cours nous avons vu qu'il y a trois alternatives possibles pour un PL : $(i)$ l'ensemble admissible peut être vide $\mathcal{D} = \emptyset$, sinon $(\mathcal{D} \neq \emptyset)$ $(ii)$ la fonction objectif n'est pas majorée (cas d'un max) ou n'est pas minorée (cas d'un min) sur $\mathcal{D}$, $(iii)$ dans le cas contraire, le PL admet une solution (pas forcément unique). Ces trois alternatives vont se rencontrer dans les exemples à résoudre.

**Utilisation de gurobi à partir de python**

Le solveur est utilisé via un module python appelé gurobipy. Nous allons nous intéresser aux dernières nouveautés qui permettent de définir un problème linéaire (ou quadratique) directement à partir de tableaux numpy et aussi de matrices creuses (plusieurs types définis dans scipy.sparse). C'est peut-être le seul solveur à proposer ces fonctionnalités qui évitent de perdre du temps à définir le modèle [2]. La documentation complète est disponible à l'url :

`https://docs.gurobi.com/projects/optimizer/en/current/reference/python.html#secpython/`

mais l'exemple suivant et les explications dans ce TP devraient être suffisants pour résoudre tous les exercices. Pour débuter regardons comment résoudre l'exercice 1 du TD1 à l'aide de gurobi :

$$\begin{array}{l} \max_{x\in\mathbb{R}^2} f(x) = 1700x_1 + 3200x_2 \\ x_1, x_2 \geq 0 \end{array}$$

$$\begin{array}{rcrcl} & & 3x_2 & \leq & 39 \\ x_1 & + & 5x_2 & \leq & 70 \\ 3x_1 & + & 2x_2 & \leq & 67 \\ 3x_1 & & & \leq & 57 \end{array}$$

Voici un script possible pour résoudre ce problème (**BIEN LIRE les commentaires !**) :

```
import numpy as np
import gurobipy as gp

# on définit le vecteur c de la fct objectif
c = np.array([1700, 3200])    #  attention un tableau 1d numpy n'est pas un tableau 2d
                              #  à une seule ligne ou une seule colonne !

# on définit la matrice A et le vecteur b des contraintes d'inégalité
A = np.array([ [ 0, 3],
               [ 1, 5],
               [ 3, 2],
               [ 3, 0]])

b = np.array([39, 70, 67, 57])  # même remarque que pour le tableau c
```

1. Une partie des variables sont continues et une autre partie entières et/ou binaires.
2. Classiquement les contraintes sont entrées une par une ; lorsqu'il y a beaucoup de contraintes, cela peut prendre du temps quand on utilise un langage interprété comme python.

```
# On définit maintenant le modèle pour gurobi.
# 1/ Il faut d'abord instancier un objet de type ''Modèle Gurobi''
m = gp.Model()

# 2/ On définit les variables du modèle (le type est ''MVar Gurobi'')
# il y a pas mal de réglages possibles (type des variables, bornes inf et/ou sup, donner un nom, etc.)
# par défaut on a des variables continues et bornées inférieurement par 0
x = m.addMVar(len(c))

# 3/ On définit la fonction objectif, exactement comme on ferait pour définir le produit scalaire
# entre deux vecteurs numpy, c'est à dire avec l'opérateur @ (attention cependant si c est bien un
# tableau numpy, x joue le rôle d'une sorte de variable formelle).
# Dans notre cas il faut préciser qu'on veut maximiser (le défaut est de minimiser)
# GRB est un sous-module de gurobipy définissant toutes les constantes.
m.setObjective(c @ x, gp.GRB.MAXIMIZE)

# 4/ Les contraintes se définissent aussi naturellement que la fonction objectif, c'est à dire
# avec l'opérateur @ (bien sûr pour des contraintes d'égalité il faudrait utiliser A @ x == b
# et on pourrait aussi utiliser A @ x >= b), on peut aussi ajouter autant de contraintes que l'on
# veut.
m.addConstr( A @ x <= b)

# 5/ reste plus qu'à résoudre, ce qui s'obtient avec :
m.optimize()

# 6/ après optimisation, on peut :

# 6-a/ obtenir le statut avec :
m.Status
# qui retourne 2 pour OPTIMAL, 3 pour INFEASIBLE, 4 pour INF_OR_UNBD  et 5 pour UNBOUNDED = 5
# mais d'autres possibilités existe comme 1 pour LOADED (le modèle n'a pas encore été optimisé)
# ou encore comme 9 pour TIME_LIMIT (il est possible de définir un temps max à ne pas
# dépasser).

# 6-b/ obtenir la solution (si le statut est OPTIMAL) sous forme de liste avec :
x_opt = m.X  # ou encore np.array(m.X) pour l'avoir sous forme de tableau numpy

# 6-c/ la valeur de la fonction objectif avec
f_opt = m.ObjVal
```

*Remarques et compléments :* (**LIRE !**)

— Le solveur sort des informations sur la console. Il est possible de les retirer en invoquant la notion d'environnement du solveur, voir :

`https://support.gurobi.com/hc/en-us/articles/360044784552-How-do-I-suppress-all-console-output-from-Gurobi-`

— Sur les options de `.addMVar(nb_vars)`. L'argument imposé est le nombre de variables et sans précision, il s'agira de variables continues inférieurement bornées par 0. Il existe (entre autres) les arguments optionnels suivants :

- `lb=, ub=` bornes inférieures et supérieures ;
- `vtype=` type des variables : `'C'`, `'I'`, `'B'`.

Pour chaque argument, vous pouvez vous contentez d'un scalaire qui sera valable pour toutes les composantes, sinon il faut un *itérable* comme une liste. Pour le problème précédent, on aurait pu utiliser :

```
A = np.array([[1, 5],
              [3, 2]])
b = np.array([70, 67])
# instantiation du modèle...
m = gp.Model()
# on donne les bornes sup lors de la définition des variables
x = m.addMVar(len(c), ub=[19,13])
```

Si vous avez certaines variables bornées supérieurement et pas d'autres, vous pouvez utiliser le nombre flottant spécial $Inf$, qui s'obtient via `np.inf` ou encore `float('inf')`, ainsi pour 3 variables avec la première majorée par 2 et les 2 suivantes non majorées, on utilisera `ub=[2,np.inf,np.inf]`.

— Le solveur effectue certains prétraitements par défaut (via un "présolveur") et cela peut le conduire ensuite à obtenir la solution sans aucune itération (voir les informations sorties par le solveur dans les exercices suivants).

— Le solveur peut travailler sur le problème dual (cf GRO 2). Ainsi dans certains cas, il ne peut pas savoir si le primal est non borné ou infaisable (domaine $\mathcal{D} = \emptyset$) d'où parfois la réponse 4 (`INF_OR_UNBD`) concernant le statut de l'optimisation. Si on veut savoir de quoi il en retourne, il est possible de refaire une optimisation du modèle en précisant :

```
m = gp.Model()
m.Params.DualReductions = 0  # pour savoir si notre problème est infaisable ou non borné
```

— Pour avoir une réponse plus explicite concernant le statut de l'optimisation effectuée par gurobi vous pourrez utiliser la fonction `status_to_str` du mini-module `util_tp1` disponible sur Arche. :

```
from util_tp1 import status_to_str
# ....
# m est un modèle gurobi
print(status_to_str(m.Status))
```

— Il est possible et fortement conseillé pour les gros problèmes d'utiliser des matrices creuses pour définir la ou les matrices des contraintes, cf exercice 3.

— gurobi s'arrête lorsqu'il a détecté une solution optimale mais il peut essayer de trouver l'ensemble des solutions (ce point n'est pas abordé dans ce TP).

— Lors de l'ajout de contraintes avec la méthode `addConstr`, si le second membre est constant (vecteur de 0, de 1, etc.), vous pouvez utiliser un scalaire, par exemple $Ax \geq 0$ peut s'écrire :

```
m.addConstr( A @ x >= 0 )
```

— Les variables gurobi peuvent être manipulées comme des tableaux numpy, on peut donc désigner une partie des variables à l'aide d'une liste d'indices ou de tranches. Exemples :

```
n = 40
m = gp.Model()  # un modèle gurobi
X = m.addMVar(n)  # un ''vecteur'' (de variables gurobi) à n composantes
X[::2] # toutes les composantes paires
X[1::2] # toutes les composantes impaires
X[:10] # les 10 premières composantes
X[-10:] # les 10 dernières composantes
X[[1,5,7]] # les composantes d'indices 1, 5 et 7
```

Ces possibilités peuvent être utilisées lorsqu'on définit la fonction objectif et les contraintes (cela peut servir pour l'exercice 3). Par exemple si la fonction objectif ne dépend que des $m$ dernières variables, on pourra utiliser :

```
# modele est un modèle gurobi
# c un ndarray 1d de profil (m,) (coefs s'appliquant aux m dernières variables)
modele.setObjective( c @ X[-m:], gp.GRB.MINIMIZE) # ou  gp.GRB.MAXIMIZE si c'est un max...
```

Si les $p$ contraintes $Ax <= b$ ne dépendent que des $m$ premières variables, on pourra se contenter de former un tableau $A$ de profil $(p, m)$ (au lieu d'un profil $(p, n)$) et écrire alors :

```
# modele est un modèle gurobi
# A un ndarray 2d de profil (p,m)
# b un ndarray 1d de profil (p,)
modele.addConstr( A @ X[:m] <= b )
```

— **Attention :** on peut déclarer les variables les unes après les autres avec `gp.addVar` (à ne pas confondre avec `gp.addMVar`) mais cela est, en général, moins efficace et moins pratique que l'approche vectorielle/matricielle utilisée dans l'énoncé.

**Exercice 1.** *Programmation linéaire*

Résoudre les problèmes suivants en utilisant le solveur gurobi :

(a) Problème de production

$$\begin{cases} \max\left[F = 50x_1 + 40x_2 + 70x_3 + 80x_4\right] \\ 2x_1 + 4x_2 + 8x_3 + 6x_4 \leq 100 \\ 10x_1 + 8x_2 + 6x_3 + 10x_4 \leq 160 \\ x_1 + x_2 + 2x_3 + 2x_4 \leq 20 \\ x_1,\, x_2,\, x_3,\, x_4 \geq 0 \end{cases}$$

(b) optimum unique à base dégénérée

$$\begin{cases} \max\left[F = x_1 + x_2\right] \\ 3x_1 + 2x_2 \leq 15 \\ 3x_1 + 4x_2 \leq 21 \\ x_2 \leq 3 \\ x_1,\, x_2 \geq 0 \end{cases}$$

(c) optimum infini

$$\begin{cases} \max\left[F = x_1 + 3x_2\right] \\ x_1 + x_2 \geq 3 \\ x_1 - 2x_2 \geq 5 \\ -2x_1 + x_2 \leq 5 \\ x_1,\, x_2 \geq 0 \end{cases}$$

(d) ensemble des solutions réalisables vide

$$\begin{cases} \max\left[F = x_1 - x_2\right] \\ x_1 + 2x_2 \leq 5 \\ 2x_1 + x_2 \leq 6 \\ x_1 + x_2 \geq 4 \\ x_1,\, x_2 \geq 0 \end{cases}$$

**Exercice 2.** *Programmation linéaire en variables* $\{0,1\}$ *– Problème de sac à dos.*

Résoudre chacun des problèmes de "sac-à-dos" suivants (ne pas oublier d'utiliser `vtype='B'` lors de l'instantiation des variables).

$$(P_1)\begin{cases} \max\left[F(x) = 6x_1 + 9x_2 + 8x_3 + 4x_4\right] \\ 4x_1 + 6x_2 + 5x_3 + 4x_4 \leq 8 \\ 3x_1 + 6x_2 + 4x_3 + 3x_4 \leq 7 \\ x_1, x_2, x_3, x_4 \in \{0,1\} \end{cases} \qquad (P_2)\begin{cases} \max\left[F(x) = 6x_1 - 9x_2 + 8x_3 + 4x_4\right] \\ 4x_1 - 6x_2 + 5x_3 - 4x_4 \leq 2 \\ 3x_1 - 6x_2 - 4x_3 + 3x_4 \leq 1 \\ x_1, x_2, x_3, x_4 \in \{0,1\} \end{cases}$$

**Exercice 3.** *Stockage (il s'agit de l'exercice 3 du TD 1).*

On dispose de $m$ clefs USB sur lesquelles on veut stocker $n$ fichiers de taille $t_1, \cdots, t_n$. Les clefs USB ont toutes la même capacité de stockage $S$. Tous les fichiers doivent être stockés. Pour qu'un stockage soit admissible il faut que la somme des tailles des fichiers stockés sur une clef soit inférieure ou égale à $S$ (on ne peut pas dépasser la capacité d'une clef...). De plus, un fichier n'est stocké qu'une seule fois sur une seule clef. Déterminer un stockage consiste alors à déterminer sur quelle clef $j$ est stocké le fichier $i$ donné. On cherche à déterminer le stockage (admissible) des fichiers sur les clefs USB de façon à minimiser le nombre de clefs utilisées.

*Modélisation* (Rappel)

On introduit les variables suivantes :

- les variables binaires $y_j$ pour indiquer si la clef USB $j$ est utilisée ($y_j = 1$) ou non ($y_j = 0$).
- les variables binaires $x_{ij}$ pour indiquer si le fichier $i$ est stocké sur la clef $j$ ($x_{ij} = 1$) ou non ($x_{ij} = 0$).

Le problème du stockage se modélise alors par le programme linéaire suivant (cf. TD1) :

$$(P)\begin{cases} \min \displaystyle\sum_{j=1}^{m} y_j \\ \displaystyle\sum_{i=1}^{n} t_i x_{ij} \leq S y_j, \quad \forall j = 1, \cdots, m \\ \displaystyle\sum_{j=1}^{m} x_{ij} = 1, \quad \forall i = 1, \cdots, n \\ y_j \in \{0,1\}, \quad \forall j = 1, \cdots, m \\ x_{ij} \in \{0,1\}, \quad \forall i = 1, \cdots, n \end{cases}$$

En regroupant les variables $x_{ij}$ et $y_j$ dans un seul vecteur $\mathbf{x}$ de $(n+1)m$ composantes comme suit :

$$\mathbf{x} = (x_{11}, x_{21}, \cdots, x_{n1} \,|\, x_{12}, x_{22}, \cdots, x_{n2} \,|\, \cdots \,|\, x_{1m}, \cdots, x_{nm} \,|\, y_1, \cdots, y_m)^\top$$

$(P)$ peut se mettre sous la forme :

$$(PL) \begin{cases} \min\limits_{\mathbf{x}} F(\mathbf{x}) = \mathbf{p}^\top \mathbf{x} \\ A\mathbf{x} \leq \mathbf{b} \\ B\mathbf{x} = \mathbf{d} \\ \mathbf{x} \in \{0,\, 1\} \end{cases}$$

La matrice $A$ est de taille $m \times (n+1)m$ (c'est-à-dire $m$ lignes et $(n+1)m$ colonnes) avec la structure suivante :

$$A = \left(\begin{array}{ccc|ccc|c|ccc|ccc} t_1 & \cdots & t_n & & 0 & & & & 0 & & -S & & 0 \\ & 0 & & t_1 & \cdots & t_n & & & 0 & & & -S & \\ & & & & & & \ddots & & & & & & \ddots \\ & 0 & & & 0 & & & t_1 & \cdots & t_n & 0 & & -S \end{array}\right)$$

Le dernier bloc diagonal avec les $-S$ est de taille $m \times m$. Les autres blocs ($m$ blocs) contiennent le vecteur $\mathbf{t} = (t_1, \cdots, t_n)$ et sont tous de taille $m \times n$.

La matrice $B$ est de taille $n \times (n+1)m$ (c'est-à-dire $n$ lignes et $(n+1)m$ colonnes) et elle a la structure suivante :

$$B = \left(\begin{array}{ccc|ccc|c|ccc|c} 1 & & & 1 & & & & 1 & & & \\ & \ddots & & & \ddots & & \cdots & & \ddots & & 0 \\ & & 1 & & & 1 & & & & 1 & \end{array}\right)$$

La matrice $B$ est composée de $m$ blocs de matrice identité de taille $n \times n$ et d'un bloc de taille $n \times m$ ne contenant que des 0.

Les vecteurs $\mathbf{p}$, $\mathbf{b}$ et $\mathbf{d}$ sont donnés par

$$\begin{aligned} \mathbf{b} &= (0, \cdots, 0)^\top \in \mathbb{R}^m \\ \mathbf{d} &= (1, \cdots, 1)^\top \in \mathbb{R}^n \\ \mathbf{p} &= (\underbrace{0, \cdots, 0}_{nm}, \underbrace{1, \cdots, 1}_{m}) \in \mathbb{R}^{(n+1)m} \end{aligned}$$

**Travail demandé**.

*Le code pourra être organisé avec 3 fichiers, (i)* `util_tp1.py` *à télécharger depuis Arche, (ii) un fichier (que vous pouvez appeler* `cles_usb.py`*) pour écrire la fonction suivante (construction des matrices $A$ et $B$ et du vecteur $p$) ainsi qu'une fonction permettant d'obtenir une solution heuristique, et (iii) un fichier (*`tests.py`*) pour résoudre les différents cas proposés dans l'énoncé. Les deux premiers ont un rôle de "bibliothèque" et le dernier un rôle de "script".*

1. Écrire une fonction python qui prend comme argument $S$, $m$ et le tableau des tailles des fichiers :

$$\mathbf{t} = [t_1,\, t_2,\, \cdots, t_n]\,.$$

et construit [3] les matrices $A$ et $B$ sous forme creuse et le vecteur $\mathbf{p}$. Aide : il est clair que ces matrices contiennent essentiellement des 0, les stocker en tant que tableaux (pleins) numpy n'est pas la solution adéquate. Les matrices creuses permettent de ne stocker que les éléments non nuls sous une certaine forme. Le module scipy.sparse offre un grand nombre de ces formats qui ont tous leurs avantages et inconvénients. Pour contruire une telle matrice un format assez souple est le format "$i, j, val$", si votre matrice à $m$ coefficients non nuls, on les stocke (dans n'importe quel ordre) à l'aide de ces 3 tableaux : pour un coefficient en supposant qu'on le considère comme le $k$-ème coefficient non nul alors $i[k]$ correspondra à son indice de ligne, $j[k]$ à son indice de colonne et $val[k]$ sa valeur. Une matrice de ce format (appelé `coo_array` dans scipy) sera obtenue avec :

3. En utilisant le "broadcasting" permis par gurobi (voir avant dernière remarque avant les exercices) il est inutile de créer les vecteurs $\mathbf{b}$ et $\mathbf{d}$ ; de plus il suffit de créer le sous-vecteur des $m$ dernières composantes de $\mathbf{p}$ (cf dernière remarque).

```
import numpy as np
import scipy.sparse as sp
......
# construction des tableaux i, j et val
......
# instantiation de la matrice creuse
A = sp.coo_array((val,(i,j)), shape=(m,n))
```

Pour construire sans effort les tableaux (`numpy.ndarray`) "$i, j, val$" permettant de définir les matrices creuses $A$ et $B$, et le tableau $\mathbf{p}$ on pourra utiliser :

(a) les fonctions `numpy.ones` et `numpy.arange` (on pourra préciser le type avec l'argument optionnel `dtype='int32'` lorsqu'on est sûr qu'il s'agit d'entiers) ;

(b) le produit de *Kronecker* de 2 vecteurs et/ou matrices qui s'obtient avec la fonction `numpy.kron`. Cas 1d : si $u$ et $v$ sont deux vecteurs avec $n_u$ et $n_v$ composantes alors : $u \otimes v$ est un vecteur à $n_u \times n_v$ composantes qui peut s'écrire par blocs :

$$u \otimes v = [\underbrace{u_1 v}_{n_v}, \underbrace{u_2 v}_{n_v}, \ldots, \underbrace{u_{n_u} v}_{n_v}]$$

(c) Vous pouvez construire la partie régulière de la matrice $A$ par la méthode "$i, j, val$" puis définir $A$ comme la concaténation de cette matrice avec (à sa droite) une matrice identité multipliée par $-S$. Une matrice creuse identité $m \times m$ s'obtient avec la fonction `sp.eye(m)`, et la concaténation à droite (appelée aussi concaténation horizontale) avec la fonction `sp.hstack` (mettre les 2 matrices à concaténer dans un tuple) où `sp` est le raccourci pour `scipy.sparse`.

2. Tester votre fonction avec les données *jouets* suivantes :

$$n = 3,\ m = 4,\ S = 4,\ t = [1, 2, 3]$$

que vous pouvez obtenir (comme les données des 5 autres tests) via un appel à :

```
# il faut avoir importé choix_test : from util_tp1 import choix_test
S, t, n, m = choix_test(1)  # 1 pour le test 1, 2 pour le test 2, etc...
```

La sortie d'une matrice creuse n'est pas toujours très lisible, vous pouvez retrouver une matrice pleine à l'aide de la méthode `toarray` (`A.toarray(), B.toarray()`) ce qui sera plus facile pour comparer avec les structures des matrices $A$ et $B$ données p. 5.

3. Écrire dans un autre fichier, un "script", qui :

(a) récupère les données $S, t, n, m$ (`choix_test(1)` pour obtenir les données *jouets* précédentes) ;

(b) construit les matrices $A$, $B$ et le vecteur $p$ correspondants aux données $S, t, n, m$ (appel à la fonction codée précédemment) ;

(c) construit le modèle pour le solveur gurobi puis le résout ;

(d) et enfin affiche la répartition des fichiers sur chaque clef. On pourra utiliser les fonctions `post_traitement` (affichage de la liste des fichiers pour chaque clé) et `printkey` fournies dans le module `util_tp1.py`, cette dernière affichant graphiquement la distribution des fichiers sur les clefs.

4. Le même script peut être utilisé pour les tests suivants (il suffit de changer le numéro du test dans l'appel à `choix_test`) :

(a) test 2 : $n = 11$, $m = 4$ et $S = 8$. $\mathbf{t} = [1, 3, 2, 1, 2, 4, 2, 3, 5, 2, 2]$.

(b) test 6 : donné par un de vos camarades, c'est un test minimal qui met en échec la méthode heuristique (cf question suivante) qui trouve 3 clés/disques au lieu de 2. $n = 6$, $m = 3$. $\mathbf{t} = [5, 5, 3, 2, 2]$.

(c) tests 3, 4 et 5 : ils sont construits aléatoirement (mais en fixant l'état du générateur aléatoire, vous allez donc résoudre les mêmes problèmes). Les tests 4 et 5 permettent d'obtenir des instances qui commencent à être un petit peu plus difficile à résoudre pour gurobi. Remarque : le test 5 dépasse les limites de l'utilisation de gurobi sans licence.

5. *Méthode heuristique.* En allant plus loin, il est vraisemblable qu'on trouvera des instances du problème qui mettront le solveur en difficulté. D'autre part, les valeurs de $m$ ont été données précédemment mais comment obtenir une valeur de $m$ pas trop grande ? La réponse (à ces deux questions) consiste à ne pas chercher une solution optimale en se contentant d'une solution "proche". Écrire une fonction python correspondant à l'heurisque expliquée dans l'exercice 3 feuille 1. Comparer avec la solution optimale trouvée précédemment (dans certains cas une solution optimale n'est pas obtenue, en particulier sur le test 6 et sur les tests 3, 4 et 5 où la solution heuristique donne une clé de plus que nécessaire. Pour vous aider voici un début possible pour cette fonction :

```
def sol_heuristique(S,t):
  n = len(t)
  # tri des fichiers (calcul des indices inds de sorte que t[inds[0]] <= t[inds[1]] <= ... t[inds[n-1]]
  inds = np.argsort(t)
  # petit test
  if t[inds[n-1]] > S :
      raise RuntimeError("La taille des clés est inférieure au plus gros fichier !")
```

6. Un souci qui peut être rencontré par un solveur dans l'exploration des possibilités est ici la multiplicité des solutions optimales (toute permutation d'une solution optimale des $m$ clés est encore optimale). De plus on aimerait que les premières clés soient occupées. Pour satisfaire cette dernière contrainte et ainsi restreindre un peu la multiplicité, on peut en fait modifier la fonction objectif, par exemple en utilisant :

$$\tilde{f}(x,y) = \sum_{i=1}^{m} i y_i$$

Pour cela il suffit de redéfinir le vecteur $p$ par :

```
p = np.arange(1,m+1, dtype='int32')
```

Refaire le même calcul sur le test 4 (et le test 5 si vous avez obtenu une licence). L'influence de cette modification était très claire avec la version précédente de gurobi (grosse accélération) avec la version actuelle, on a mesuré les temps suivants :

- sur ma machine (pas très récente) :
  - pour le test 4 je passe de 2.75 s (avec $f$) à 0.2 s (en utilisant $\tilde{f}$) donc bien plus rapide ;
  - pour le test 5 je passe de 4,78 s à 5.17 s donc un peu plus lent !
- sur une des machines de l'école (test 4) je passe de 0.32 s (avec $f$) à 0.6 s (en utilisant $\tilde{f}$)
- sur un portable mac récent (celui de mon collègue Jean-François) :
  - test 4 : 0.08 s (avec $f$) à 0.38 s (avec $\tilde{f}$) ;
  - test 5 : 2.49 s (avec $f$) à 3.99 s (avec $\tilde{f}$).

Les résultats sont donc mitigés. Il faudrait faire sans doute plus de tests pour avoir une idée mais la tendance semble être que, sur ce problème précis, $\tilde{f}$ (qui enlève une partie des solutions optimales) ne facilite plus la recherche de la solution avec la version actuelle de gurobi voire la ralentisse !

7. Que faut-il changer dans la modélisation PL pour avoir deux fois chaque fichier sur deux clés différentes ? Remarque : il faudrait aussi adapter l'heuristique.